

Relational Database Design

Practical Notes — From Data Modeling to SQL Normalization & JOINS

Course topic	Relational Database Design (PostgreSQL / pg4e style)
Focus	Data modeling, keys, normalization, SQL CREATE/INSERT/JOIN
Format	Practical, example-driven (music database: track/album/artist/genre)

Table of Contents

- 1. Why Relational Database Design Matters
- 2. The Core Idea: Compression Through Connection
- 3. Building a Data Model — Step by Step (Music Example)
- 4. Three Kinds of Keys: Primary, Logical, Foreign
- 5. From Logical Model to Physical Model
- 6. Writing SQL: Creating Tables
- 7. Inserting Normalized Data
- 8. Reconstructing Data with JOIN
- 9. ON DELETE Rules: CASCADE, RESTRICT, SET NULL
- 10. Quick Reference Cheat Sheet
- 11. Practice Tasks (10 Hands-On Exercises)

1. Why Relational Database Design Matters

Database design is more of an **artistry** than a fixed formula — a practical skill you improve at by doing it repeatedly, ideally with other people around a whiteboard. The end goal of design is always the same: draw a picture of the **shape of your data**.

In this picture, each box is a **table** and each line connecting boxes is a **foreign key**. In large real-world projects, this diagram (the *data model*) is often treated as the soul of the application — sometimes literally printed and pinned on a wall.

■ **Practical takeaway:** Before writing a single line of SQL, sketch your tables and the arrows between them. The SQL is just a mechanical translation of that picture.

2. The Core Idea: Compression Through Connection

The single most important rule in database design: **never store the same string more than once**. This is called avoiding **vertical replication of string data**.

The Problem — Example

Imagine a spreadsheet of your music collection. Every time you add a track by Black Sabbath, you retype "Black Sabbath" again. If you ever have a typo, you now must hunt down and fix it in every single row.

The Fix

Store the string exactly **once** in its own table, give that row a numeric ID, and reference that ID everywhere else instead of retyping the string.

```
Artist table:  
id = 42 name = 'Black Sabbath'
```

```
Album / Track tables reference it as:  
artist_id = 42, 42, 42, 42, 42 ...
```

Fix a typo **ONCE** in the artist table -> every reference updates automatically.

■ **Numbers are cheap to repeat; strings are expensive to repeat. Numbers (ratings, counts, lengths) can live directly as columns. Repeated text (artist names, album titles, genres) must be pulled into their own table.**

Important: this doesn't mean the user interface is wrong for showing repeated text (e.g. a sortable list with "Rock", "Rock", "Metal" repeated). The UI can and should look this way — it's convenient for users. We simply **don't store it that way**; we reconstruct it later using JOIN (Section 8).

3. Building a Data Model — Step by Step (Music Example)

Goal: track music — tracks, albums, artists, genres, ratings, play counts.

Step 1 — Pick the central object

In multi-user systems, the natural center is usually the **user**. This app is single-user, so instead ask: *what is the one thing this application organizes?* Here, that's the **track**.

Step 2 — Attach simple numeric attributes directly

Numbers don't need their own table because they aren't vertically replicated in a costly way. So **length**, **rating**, and **play count** become plain columns on the track table, along with the track's **title**.

Step 3 — Split out anything that repeats as text

- **Album** — many tracks belong to one album → own table
- **Artist** — many albums belong to one artist → own table
- **Genre** — repeated text describing style → own table

Step 4 — Decide where each relationship connects

Genre could connect to track, album, or artist — and the choice changes behavior:

- Connect genre to **track** → changing one track's genre affects only that track.
- Connect genre to **album** → changing it changes every track on that album at once.
- Connect genre to **artist** → changing it changes every track by that artist at once.

For this music app, the right choice is **track** → **genre** (a track's genre is independent).

Resulting Logical Model

```
track --(many-to-one)--> album --(many-to-one)--> artist
track --(many-to-one)--> genre
```

```
track columns: title, length, rating, count
album columns: title
artist columns: name
genre columns: name
```

■ **Rule of thumb:** if a column value is text that repeats across many rows, it becomes its own table. If it's a number (or otherwise cheap/unique per row), keep it as a plain column.

4. Three Kinds of Keys: Primary, Logical, Foreign

Key Type	Purpose	Example	Data Type
Primary key	Unique internal handle for a row; used for joining & updating cheaply.	album.id	Integer (auto-increment)
Logical key	What the outside world / users actually search by	track title, email address, artist name	String (VARCHAR), UNIQUE
Foreign key	Integer in one table pointing to the primary key in another table	track.album_id = album.id	Integer, REFERENCES

Naming convention used throughout these notes

- Primary key column is always named **id**.
- Foreign key column = **<referenced_table>_id** (e.g. **artist_id** points to the artist table).
- Consistency matters more than which convention you pick — follow your organization's rules.

■ **Never use a logical key (like an email address) as your primary key. Logical keys can change (users get married, fix typos in their name); primary keys must never change. Also, integer comparisons are far faster than string comparisons — this is what makes databases fast at scale.**

Avoid GUIDs as primary/foreign keys when possible too — they work, but are less efficient than plain auto-incrementing integers for joining.

5. From Logical Model to Physical Model

Turning the picture into real tables is mechanical once you know the pattern:

- Pick the **logical key** (the searchable string field) for each table.
- Add a **primary key** (auto-incrementing integer **id**) to every table.
- For every arrow in your diagram, add a **foreign key** column at the "many" end pointing to the "one" end's id.

Example: 4 tables (track, album, artist, genre) with 3 arrows between them produces 4 primary keys, 4 logical keys, and 3 foreign keys total.

Many-to-one relationship

"Many tracks belong to one album" — on the **one** side (album), we just have the primary key. On the **many** side (track), we add a foreign key column: **album_id**.

6. Writing SQL: Creating Tables (PostgreSQL)

Postgres's **SERIAL** type auto-generates an incrementing integer — perfect for primary keys. **UNIQUE** marks logical keys and tells the database to build a fast index for lookups.

Pattern to memorize

```
id SERIAL PRIMARY KEY -- pattern for every primary key
name VARCHAR(128) UNIQUE -- pattern for a string logical key
```

Full example — music database

```
CREATE TABLE artist (
  id SERIAL,
  name VARCHAR(128) UNIQUE,
  PRIMARY KEY(id)
);

CREATE TABLE album (
  id SERIAL,
  title VARCHAR(128) UNIQUE,
  artist_id INTEGER REFERENCES artist(id) ON DELETE CASCADE,
  PRIMARY KEY(id)
);

CREATE TABLE genre (
  id SERIAL,
  name VARCHAR(128) UNIQUE,
  PRIMARY KEY(id)
);

CREATE TABLE track (
  id SERIAL,
  title VARCHAR(128),
  len INTEGER,
  rating INTEGER,
  count INTEGER,
  album_id INTEGER REFERENCES album(id) ON DELETE CASCADE,
  genre_id INTEGER REFERENCES genre(id) ON DELETE CASCADE,
  PRIMARY KEY(id),
  UNIQUE(title, album_id)
);
```

■ On the track table, title alone **CANNOT** be **UNIQUE** (many albums can have a track called "Moonlight"). Instead we use a composite **UNIQUE(title, album_id)** — the combination must be unique, not each column alone.

You can verify what Postgres actually built with the **\d tablename** command in psql — it shows column types, the unique constraint (B-tree index), and the foreign key definitions.

7. Inserting Normalized Data

Because **id** is SERIAL, you never insert it manually — the database assigns it automatically, starting at 1 and incrementing.

```
INSERT INTO artist (name) VALUES ('Led Zeppelin'); -- gets id = 1
INSERT INTO artist (name) VALUES ('AC/DC'); -- gets id = 2

INSERT INTO album (title, artist_id) VALUES ('Who Made Who', 2); -- AC/DC's id
INSERT INTO album (title, artist_id) VALUES ('IV', 1); -- Led Zeppelin's id

INSERT INTO genre (name) VALUES ('Rock'); -- id = 1
INSERT INTO genre (name) VALUES ('Metal'); -- id = 2

INSERT INTO track (title, len, rating, count, album_id, genre_id)
VALUES ('Black Dog', 296, 5, 3, 2, 1);
```

Work from the **outside tables inward**: insert artists first, then albums (which need `artist_id`), then genres, and finally tracks (which need both `album_id` and `genre_id`). Keep a scratch note of which name maps to which id as you go — this is exactly how it's done by hand before you learn shortcuts.

■ All this bookkeeping exists for one reason: to store each string exactly once and reference it by number everywhere else.

8. Reconstructing Data with JOIN

Once data is normalized (spread across tables with no repeated strings), we need a way to put it back together for the user interface. That tool is **JOIN**.

INNER JOIN — matching rows only

```
SELECT album.title, artist.name
FROM album JOIN artist
ON album.artist_id = artist.id;
```

The **ON** clause tells the database how rows connect — follow the foreign key to the primary key it references.

CROSS JOIN — all combinations (rarely what you want)

```
SELECT * FROM track CROSS JOIN genre;
-- 4 tracks x 2 genres = 8 rows, including combinations that don't logically match
```

Think of INNER JOIN as a CROSS JOIN followed by a WHERE filter that keeps only the rows where the ON condition is true. CROSS JOIN is expensive at scale (millions x millions of rows) — avoid it in real queries.

Multi-table JOIN — full reconstruction

```
SELECT track.title, artist.name, album.title, genre.name
FROM track
JOIN genre ON track.genre_id = genre.id
JOIN album ON track.album_id = album.id
JOIN artist ON album.artist_id = artist.id;
```

■ This query reconstructs the exact vertical replication ("Rock, Rock, Metal, Metal...") the UI needs — but the repeated text is never stored that way. It's generated only at query time, on the fly.

9. ON DELETE Rules: CASCADE, RESTRICT, SET NULL

When a parent row (e.g. a genre) is deleted, what happens to child rows (tracks) that reference it? You declare this behavior on the foreign key.

Option	Behavior
ON DELETE CASCADE	Deleting the parent automatically deletes all matching child rows too. Keeps the data model clean and
ON DELETE RESTRICT	Blocks the delete entirely if child rows still reference the parent. The DELETE statement fails/errors.
ON DELETE SET NULL	Child rows are kept, but their foreign key column is set to NULL. Requires the foreign key column to al

When SQL "blows up" on a delete, it's usually because you (or a past teammate) told it to enforce a rule via RESTRICT. That's a feature, not a bug — it protects data consistency.

10. Quick Reference Cheat Sheet

Concept	Rule of Thumb
Vertical replication	If a text value repeats across rows -> move it to its own table.
Primary key	SERIAL PRIMARY KEY on an `id` column. Never expose this as a user-facing search field.
Logical key	VARCHAR ... UNIQUE. What users search / see.
Foreign key	INTEGER REFERENCES other_table(id) ON DELETE ...
Naming	`<table>_id` for every foreign key column.
Many-to-one	Foreign key goes on the 'many' side, pointing to the 'one' side's id.
JOIN	Use INNER JOIN + ON to reconnect normalized tables back into readable rows.
Composite UNIQUE	UNIQUE(title, album_id) when a single column can't be unique alone.

11. Practice Tasks — 10 Hands-On Exercises

Do these in order. Use PostgreSQL (psql or pgAdmin). Each task builds on the previous one, just like the music database example.

Task 1 — Design a logical model

Pick a domain you know (e.g. a bookstore: book, author, publisher, category). On paper, draw boxes for each object and arrows showing many-to-one relationships. Identify which fields are text (candidates for their own table) vs. numbers (plain columns).

Task 2 — Identify vertical replication

Take a spreadsheet or CSV you have (or invent one with 10 rows) that repeats a text value (e.g. category name) across many rows. Circle every repeated string and decide which table it should move into.

Task 3 — Classify the three key types

For your bookstore model, list out for every table: the primary key, the logical key, and any foreign key(s). Write them in a 3-column table like the one in Section 4.

Task 4 — Write CREATE TABLE statements

Convert your bookstore logical model into real SQL. Use SERIAL PRIMARY KEY for ids, UNIQUE for logical keys, and INTEGER REFERENCES ... ON DELETE CASCADE for every foreign key.

Task 5 — Handle a tricky UNIQUE constraint

In your book table, a title alone might not be unique (multiple editions/publishers can share a title). Add a composite UNIQUE constraint that makes sense, like `UNIQUE(title, publisher_id)`.

Task 6 — Insert data by hand

Insert 3 authors, 3 publishers, 3 categories, then 5 books referencing them. Keep a scratch note of which name maps to which generated id as you insert, exactly like the video's method.

Task 7 — Write an INNER JOIN

Write a query that joins book with author to show the book title and author name together (no ids visible).

Task 8 — Write a full multi-table JOIN

Extend Task 7 into a single SELECT that joins book, author, publisher, and category together, reconstructing every human-readable column in one row per book.

Task 9 — Experiment with ON DELETE behavior

Create a test row, then try deleting its parent three ways in three throwaway tables: one with ON DELETE CASCADE, one with ON DELETE RESTRICT, and one with ON DELETE SET NULL (remember to make the foreign key column nullable for the last one). Observe and write down what happens each time.

Task 10 — Compare CROSS JOIN vs INNER JOIN

Run a CROSS JOIN between two of your small tables and count the rows returned. Then add an ON clause to turn it into an INNER JOIN and compare the row count and results. Explain in 2-3 sentences why CROSS JOIN would be dangerous on tables with millions of rows.

End of notes. Work through the tasks in a real Postgres database (e.g. the pg4e-style setup) to make the concepts stick — reading alone won't build the instinct; running the SQL will.